

Индивидуальный проект. Этап 2

Алгоритмы решения задачи: Образование планетной системы.

Тойчубекова А.Н Четвергова М.В Просина К.М.
Чигладзе М.В. Митрофанов Т.А.

Содержание

1	Общая структура алгоритма	5
2	Инициализация системы	6
2.1	Генерация начальных положений	6
2.2	Начальные скорости (кеплеровское вращение)	7
3	Вычисление сил	9
3.1	Гравитационное взаимодействие	9
3.2	Упругое отталкивание (при контакте)	10
3.3	Сила трения (тангенциальная)	11
4	Интегрирование уравнений движения (алгоритм Верле)	13
4.1	Скоростной алгоритм Верле	13
5	Обработка слияния частиц (аккреция)	15
6	Алгоритм ускорение вычислений	17
6.1	Метод «частица–сетка» (Particle-Mesh)	17
6.2	Быстрое мультипольное разложение (FMM)	18
7	Выбор шага по времени	20
8	Контроль правильности и вывод результатов	21
9	Общая блок-схема алгоритма	22
10	Заключение	23

Список иллюстраций

Список таблиц

1 Общая структура алгоритма

Моделирование эволюции протопланетного диска осуществляется методом пошагового численного интегрирования уравнений движения системы из N взаимодействующих частиц.

Временная эволюция системы разбивается на дискретные шаги длиной Δt . На каждом шаге последовательно выполняются вычисления сил взаимодействия, обновление кинематических параметров частиц и обработка столкновений.

Общая схема алгоритма на один шаг по времени:

1. Вычисление всех сил, действующих на каждую частицу (гравитация, упругое отталкивание, трение).
2. Интегрирование уравнений движения – обновление положений и скоростей.
3. Обработка столкновений – проверка условий слияния (аккреции) и объединение частиц.
4. Обновление вспомогательных структур (дерева, сетки для ускорения вычислений).
5. Контроль сохранения величин (энергия, момент импульса) и вывод данных.

Алгоритм повторяется до достижения заданного физического времени либо до стабилизации системы.

2 Инициализация системы

Прежде чем запускать расчет, нужно смоделировать начальный газопылевой диск.

2.1 Генерация начальных положений

Мы считаем, что в начальный момент частицы равномерно заполняют круг радиуса R_0 (протопланетный диск). Чтобы получить случайную точку внутри круга, используем стандартный прием:

- генерируем два случайных числа ξ_1 и ξ_2 в интервале $(0, 1]$;
- радиус точки берем как

$$r = R_0 \cdot \sqrt{\xi_1}$$

— квадратный корень нужен для того, чтобы плотность точек была постоянной по площади; * азимутальный угол

$$\alpha = 2\pi \cdot \xi_2;$$

* координаты $x=r\cos\alpha, y=r\sin\alpha$.

В трехмерном варианте добавляем небольшой разброс по вертикали z , чтобы диск имел малую толщину. Все эти операции выполняются в цикле по всем частицам.

Алгоритм: Задаем количество частиц N и толщину диска H.

```
for i = 1..N:
    xi1 = random(0,1]
    xi2 = random(0,1]

    r = R0 * sqrt(xi1)
    alpha = 2*pi * xi2

    x[i] = r * cos(alpha)
    y[i] = r * sin(alpha)
    z[i] = (random(0,1) - 0.5) * H
```

2.2 Начальные скорости (кеплеровское вращение)

Диск вращается вокруг центрального тела (прото-Солнца). Скорость каждой частицы должна быть такой, как если бы она двигалась по круговой орбите под действием гравитации центральной массы. Согласно третьему закону Кеплера, угловая скорость ω падает с расстоянием

$$\omega(r) = \omega_0 * (R_0/r)^3/2.$$

В декартовых координатах для частицы в точке (x,y) это дает:

$$v_x = -\omega(r) \cdot y = -y \cdot \omega_0 * (R_0/r)^3/2$$

$$v_y = \omega(r) \cdot x = x \cdot \omega_0 * (R_0/r)^3/2$$

$$v_z = 0$$

Таким образом, каждая частица «нацелена» на круговое движение, но из-за

взаимного притяжения между частицами орбиты будут возмущаться.

Алгоритм:

```
for i = 1..N:  
    r = sqrt(x[i]^2 + y[i]^2)  
    omega = omega0 * (R0 / r)^(3/2)  
  
    vx[i] = -omega * y[i]  
    vy[i] = omega * x[i]  
    vz[i] = 0
```


3 Вычисление сил

На каждую частицу действуют три типа сил: гравитационное притяжение со стороны всех остальных, упругое отталкивание при соприкосновении и сила трения (диссипация). Рассмотрим каждый тип по порядку.

3.1 Гравитационное взаимодействие

Гравитационная сила между двумя частицами вычисляется по формуле Ньютона:

$$(\mathbf{F})_i^{(grav)} = \gamma m_i \sum_{j \neq i} (m_j ((\mathbf{r})_j - (\mathbf{r})_i)) / (r_{ij})^3$$

В программе это реализуется двойным циклом по всем парам частиц. Для каждой пары находим вектор разности координат, его длину, затем вычисляем силу и добавляем её к результирующей силе каждой из двух частиц. Чтобы избежать численного взрыва при очень малых расстояниях, в знаменатель добавляется малая величина «сглаживания» ε (softening).

Алгоритм ($O(N^2)$):

```
for i = 1..N:
    F[i] = (0,0,0)

    for j = 1..N, j != i:
```

```

dx = x[j] - x[i]
dy = y[j] - y[i]
dz = z[j] - z[i]

r2 = dx^2 + dy^2 + dz^2 + eps
r = sqrt(r2)

Fmag = G * m[i] * m[j] / (r2 * r)

F[i].x += Fmag * dx
F[i].y += Fmag * dy
F[i].z += Fmag * dz

```

3.2 Упругое отталкивание (при контакте)

a — это сумма радиусов двух частиц:

$$a = R_i + R_j$$

b — это текущее расстояние между центрами частиц:

$$b = r_{ij} = |r_j - r_i|$$

Если $b < a$, значит частицы проникли друг в друга (их радиусы перекрываются). В этом случае включается сила упругого отталкивания, чтобы развести их.

$$(\mathbf{F})_{(ij)}^{rep} = k[((a)/(b))^8 - 1]((\mathbf{r})_{(ij)})/(b)$$

Показатель степени 8 выбран, чтобы сила была заметна только при реальном контакте. Эта сила направлена вдоль линии, соединяющей центры. В программе

для каждой перекрывающейся пары вычисляется коэффициент, затем сила добавляется к одной частице и вычитается из другой.

Алгоритм:

```
for all pairs (i,j), i < j:
    dx = x[j] - x[i]
    dy = y[j] - y[i]
    dz = z[j] - z[i]

    b = sqrt(dx^2 + dy^2 + dz^2)
    a = R[i] + R[j]

    if b < a:
        factor = k * ((a / b)^8 - 1) / b

        Fx = factor * dx
        Fy = factor * dy
        Fz = factor * dz

        F[i] += (Fx, Fy, Fz)
        F[j] -= (Fx, Fy, Fz)
```

3.3 Сила трения (тангенциальная)

Даже если частицы столкнулись, они могут не просто отскочить, а потерять часть энергии на нагрев. Для моделирования трения используется тангенциальная сила, перпендикулярная линии центров. Её величина пропорциональна силе отталкивания и относительной тангенциальной скорости (с учётом вращения самих частиц). Коэффициент β задает интенсивность диссипации.

$$(\mathbf{F})_{ij}^{(fric)} = \beta W_{\perp} F^{(rep)}(b)(\mathbf{n})$$

где $W_{\perp}$ – относительная тангенциальная скорость с учётом вращения частиц.

Эта сила также создаёт момент, раскручивающий или замедляющий вращение каждой частицы вокруг собственной оси.

Алгоритм:

```
for all pairs (i,j), i < j:
    if distance(i,j) < (R[i] + R[j]):

        W = v[i] - v[j]

        n_norm = normalize(r[j] - r[i])
        n_tan = normalize(perpendicular(W, n_norm))

        W_perp = dot(W, n_tan)

        mu = beta * W_perp
        Ffric = mu * |Frep| * n_tan

        F[i] += Ffric
        F[j] -= Ffric
```

4 Интегрирование уравнений движения (алгоритм Верле)

Второй закон Ньютона — дифференциальное уравнение. Аналитически решить его для тысячи частиц невозможно, поэтому мы разбиваем время на малые шаги Δt и используем численный метод. Самым подходящим для гравитационных задач является алгоритм Верле (в скоростной версии).

4.1 Скоростной алгоритм Верле

Пусть в момент t_n нам известны положения $\mathbf{r}_n$, скорости $\mathbf{v}_n$ и ускорения $\mathbf{a}_n$ (ускорения вычислены по текущим положениям через сумму сил). Тогда:

1. Предсказание новых положений (используем текущую скорость и ускорение):

$$(\mathbf{r})_{(n+1)} = (\mathbf{r})_n + (\mathbf{v})_n \Delta t + 1/2(\mathbf{a})_n (\Delta t)^2.$$

2. Вычисление новых ускорений $\mathbf{a}_{n+1}$ по только что полученным положениям $\mathbf{r}_{n+1}$ (снова вызываем подпрограмму расчёта всех сил).
3. Коррекция скоростей (усредняем старое и новое ускорения):

$$(\mathbf{v})_{(n+1)} = (\mathbf{v})_n + 1/2((\mathbf{a})_n + (\mathbf{a})_{(n+1)}) \Delta t.$$

Аналогично обновляются угловые скорости ω_i с использованием моментов сил. Этот метод обладает хорошей устойчивостью и сохраняет энергию даже при относительно больших Δt , что важно для длительных расчётов.

Алгоритм:

```
for i = 1..N:
    r_new[i] = r[i] + v[i]*dt + 0.5*a[i]*dt^2

a_new = compute_acceleration(r_new)

for i = 1..N:
    v_new[i] = v[i] + 0.5*(a[i] + a_new[i])*dt
```

обновить ω (вращение) аналогично через момент ускорения

5 Обработка слияния частиц (аккреция)

Когда две частицы сближаются на расстояние меньше суммы их радиусов, они могут не просто оттолкнуться, а слиться в одну. В нашей модели слияние происходит, если выполнено условие контакта и относительная скорость не превышает порога (чтобы не моделировать разрушение). При слиянии:

- Новая частица получает суммарную массу и объём (радиус вычисляется из условия сохранения объёма).
- Её положение и скорость определяются как центр масс.
- Угловая скорость новой частицы находится из закона сохранения полного момента импульса (орбитального и собственного).

В программе после каждого шага интегрирования просматриваются все пары частиц. Если обнаружено перекрытие, пара помечается на слияние. Чтобы не допустить «многоженства» (частица не может слиться сразу с двумя за один шаг), обычно сортируют пары по расстоянию и обрабатывают их в порядке возрастания, либо объединяют все сразу с последующей перенумерацией.

- Масса:

$$m_{(new)} = m_i + m_j$$

- Радиус:

$$R_{(new)} = \sqrt{(R_i^3 + R_j^3)}$$

- Положение (ЦМ):

$$(\mathbf{r})_{(new)} = (m_i(\mathbf{r})_i + m_j(\mathbf{r})_j)/m_{(new)}$$

- Скорость (ЦМ):

$$(\mathbf{v})_{(new)} = (m_i(\mathbf{v})_i + m_j(\mathbf{v})_j)/m_{(new)}$$

- Угловая скорость: из сохранения полного момента импульса.

Алгоритм:

```
for all pairs (i,j), i < j:
    if distance(i,j) < (R[i] + R[j]):

        if |v[i] - v[j]| < v_threshold:

            new.mass = m[i] + m[j]
            new.position = center_of_mass(i, j)
            new.velocity = momentum_conservation(i, j)

            remove particles i, j
            add particle new
```


6 Алгоритм ускорение вычислений

Для больших N прямое суммирование $O(N^2)$ становится неприемлемым. Используются два подхода.

Прямой перебор всех пар даёт сложность $O(N^2)$. При $N=10^4$ это уже 108108 операций на шаг, что очень медленно. Для ускорения применяют специальные методы.

6.1 Метод «частица–сетка» (Particle-Mesh)

Идея: заменить попарные силы решением уравнения Пуассона для гравитационного потенциала на равномерной сетке.

Алгоритм:

- Распределяем массы частиц по узлам сетки (например, каждая частица «размазывается» на ближайшие 4 узла в 2D).
- Быстрым преобразованием Фурье решаем уравнение

$$\nabla^2 \Phi = 4\pi\gamma\rho$$

в частотной области.

- Находим силу как градиент потенциала на сетке.
- Интерполируем силу обратно на частицы.

Сложность снижается до

$$O(N + M \log M)$$

где M — число узлов сетки. Метод хорош для равномерных распределений, но теряет точность в плотных сгустках.

Алгоритм:

```
initialize grid

for each particle p:
    distribute mass to grid nodes

rho_hat = FFT(grid)
phi_hat = -4*pi*G * rho_hat / k^2
phi = IFFT(phi_hat)

F_grid = gradient(phi)

for each particle p:
    F[p] = interpolate(F_grid, p.position)
```

6.2 Быстрое мультипольное разложение (FMM)

Этот метод считается «золотым стандартом» для гравитационных N-тел.

Идея:

- Пространство рекурсивно разбивается на ячейки (квадродерево в 2D, октодерево в 3D).
- Для каждой ячейки вычисляются мультипольные моменты (суммарная масса, диполь, квадруполь...).

- При вычислении силы на частицу:
- если ячейка далеко — её вклад аппроксимируется несколькими первыми моментами;
- если ячейка близко — раскрываем её на дочерние ячейки, вплоть до отдельных частиц.

Сложность FM составляет $O(N)$ или $O(N \log N)$, причём метод сохраняет высокую точность и хорошо работает при сильной кластеризации — как раз в нашем случае, когда из диска формируются протопланеты.

Алгоритм верхнего уровня:

```
build_tree(particles)

function compute_force(particle, cell):
    if cell is far:
        use multipole approximation
    else:
        if cell is leaf:
            compute direct forces
        else:
            for child in cell:
                compute_force(particle, child)
```

7 Выбор шага по времени

Шаг Δt должен быть достаточно мал, чтобы:

- частица за один шаг не пролетала больше своего радиуса;
- угол поворота по орбите был небольшим (обычно $\Delta t < 0.01 T_{\min}$, где $T_{\min}$ — самый короткий период обращения в системе);
- в процессе контактного взаимодействия не происходило чрезмерного сжатия.

На практике выбирают адаптивный шаг: в начале расчёта вычисляют минимальный орбитальный период среди всех частиц и минимальное время контакта, затем берут наименьшее из этих значений, умноженное на коэффициент надёжности (например, 0.1). В ходе моделирования шаг можно пересчитывать, если условия меняются.

• Период самого быстрого обращения: $\Delta t < 0.01 T_{\min}$. • Время контакта при отталкивании: $\Delta t < 0.1 \sqrt{m/k}$.

8 Контроль правильности и вывод результатов

Чтобы убедиться, что модель работает физически, на каждом шаге (или через несколько шагов) вычисляют:

- полную механическую энергию (сумма кинетической, вращательной, гравитационной потенциальной и энергии упругого отталкивания);
- полный момент импульса;
- количество частиц, максимальную массу, размеры сгустков.

Если энергия начинает резко расти или падать — это признак численной неустойчивости, нужно уменьшить Δt .

Все эти величины, а также координаты частиц записываются в файл для последующей визуализации и анализа.

9 Общая блок-схема алгоритма

1. Задать параметры: N , R_0 , ω_0 , массы, плотность, коэффициенты k , β .
2. Сгенерировать начальные положения (равномерно по диску).
3. Задать кеплеровские скорости.
4. Вычислить начальные ускорения (гравитация + контакты).

Повторять, пока $t < t_{\max}$:

5. Для каждой частицы: предсказать новое положение по скоростному Верле.
6. По новым положениям вычислить все ускорения.
7. Обновить скорости и угловые скорости.
8. Обработать слияния (объединить перекрывающиеся частицы).
9. Перестроить вспомогательные структуры (дерево FMM или сетку).
10. $t = t + \Delta t$.
11. Если шаг кратен заданному интервалу: записать энергию, момент импульса, координаты.

10 Заключение

На втором этапе проекта разработана полная схема численного моделирования, включающая:

- инициализацию диска с кеплеровским вращением;
- вычисление гравитационных, контактных и диссипативных сил;
- интегрирование по скоростному алгоритму Верле;
- процедуру аккреции (слияния) частиц;
- методы ускорения (Particle-Mesh или FMM) для работы с большим числом частиц.